



THE UNIVERSITY OF QUEENSLAND
AUSTRALIA

Reinforcement Learning for Control of Robotic Perception Systems

Santiago Rodrigues

Specify your current degrees



Candidate's ORCID

*A thesis submitted for the degree of Bachelors of Engineering
(Honours) and Computer Science at
The University of Queensland in 2025*

METR4911

Abstract

Start this section on a new page [this template will automatically handle this].

The abstract should outline the main approach and findings of the thesis and normally must be between 300 and 800 words.

Declaration by Author

This thesis is composed of my original work, and contains no material previously published or written by another person, or generated by artificial intelligence (AI), except where clearly referenced and acknowledged either in the main text or the preliminary pages of this thesis. I have clearly stated the contribution of others to jointly authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, use of AI tools, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Use of Artificial Intelligence

I acknowledge the use of generative AI tools in completing this assignment. Details of which tools were used and how they were used are provided in the table below, along with appropriate in-text and full references. I take responsibility for critically evaluating and integrating the AI-generated content, and ensuring it adheres to academic integrity standards.

To ensure transparency and uphold academic integrity, please disclose any use of Artificial Intelligence (AI) tools in your research and thesis preparation using the following table:

AI Tool	Purpose of Use	Prompts Used	Thesis Section
{Name and version of AI tool}	{How you specifically used the AI tool, e.g., data analysis, literature review assistance, language editing}	{Exact prompts or queries you input into the AI tool}	{Specific chapters or sections affected}

How to complete your AI declaration table:

1. AI Tool: Clearly identify the name and version of each AI tool you used (e.g., ChatGPT 4.0, Grammarly).
2. Purpose of Use: Describe how you used the AI tool in your research or writing process.
3. Prompts Used: Provide the prompts or instructions you gave to the AI tool.
4. Thesis Section: Specify the location in your thesis where you used AI-generated content or assistance.

Example of a Completed AI Declaration Table:

Additional important considerations:

- You are accountable for all content included in your thesis, including material generated or assisted by AI. Ensure that all AI-generated content is accurate and original.
- Never input sensitive or confidential data into AI tools due to privacy and security risks.
- Do not credit AI tools as authors. Clearly acknowledge their assistance as outlined above.
- Some publishers do not permit AI-generated or AI-assisted text, so this should be considered if the chapter is intended for publication.

If AI was not used in your research or thesis preparation, then state “No artificial intelligence use” under this heading.

AI Tool	Purpose of Use	Prompts Used	Thesis Section
Microsoft Copilot	Improved clarity and conciseness of the abstract and introduction sections.	“Rewrite the following abstract and introduction for greater conciseness and clarity.”	Abstract and Introduction (pp. i-24)
ChatGPT 4.0	Assisted in brainstorming and refining research questions.	“Generate research questions related to sustainable urban development.”	Chapter 1: Introduction (pp. 3-5)
Elicit	Curated and summarised relevant research papers, identifying key themes.	“Summarise recent studies on sustainable urban development.”	Chapter 2: Literature Review (pp. 15-30)
IBM Watson Studio	Processed large datasets to recognise patterns and build predictive models.	N/A (data input and model training)	Chapter 4: Data Analysis (pp. 50-70)
Grammarly	Enhanced grammar, style, and academic tone throughout the thesis.	N/A (automated grammar and style suggestions)	Entire thesis
Syntho	Generated synthetic datasets to address data scarcity and preserve privacy.	N/A (synthetic data generation)	Chapter 3: Methodology (pp. 35-40)

Acknowledgments

I would like to acknowledge the contributions of my two supervisors Dr Kasra Khosoussi and Associate Professor Jen Jen Chung. They created the initial thesis topic and resources to start this project. Importantly they also provided weekly mentorship, feedback and guidance throughout this thesis.

I would also like to acknowledge, Nico Messikommer, Giovanni Cioffi, Mathias Gehrig and Davide Scaramuzza for their work on “Reinforcement Learning Meets Visual Odometry” which formed the basis of this research.

Contents

List of Figures	viii
List of Tables	ix
Chapter 1: Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Problem Statement	2
1.4 Research Objectives	2
1.5 Contributions	2
Chapter 2: Literature Review	4
2.1 LiDAR SLAM Foundations	4
2.2 Hyperparameter Tuning Strategies	4
2.3 Reinforcement Learning for Perception Control	5
2.4 Gap Analysis	5
Chapter 3: Methodology	6
3.1 SC-LIO-SAM Pipeline	6
3.2 ROS Replay and Integration	6
3.3 Reinforcement Learning Environment	8
3.3.1 Observation Design	8
3.3.2 Action Channel and Safety	8
3.3.3 Reward Shaping	8
3.4 Hyperparameter Sweep Methodology	9
3.5 PPO Training Loop	10
Chapter 4: Results	12
4.1 Hyperparameter Sweep Outcomes	12
4.2 PPO Training Performance	13
4.3 Reward Exploitation Patterns	13
4.4 Hardware Variability Observations	14

Chapter 5: Discussion	15
5.1 Policy Behaviour and Plateaued Learning	15
5.2 Reward Shaping Implications	15
5.3 SC-LIO-SAM Sensitivity and Interpretation	16
5.4 Training Stability and Variability	16
Chapter 6: Conclusion	17
6.1 Summary	17
6.2 Future Work	17
References	18
Appendix A	20
Title of Appendix A	20
Appendix B	21
Title of Appendix B	21

List of Figures

1	ROS-native replay stack linking MulRan data, SC-LIO-SAM, and the RL environment. .	7
2	Spearman correlations from the automated sweep, highlighting <code>mapping_surf_leaf_size</code> as the most influential runtime parameter.	10
3	PPO training pipeline showing the interaction between reinforcement learning components and the ROS-native SC-LIO-SAM environment.	11
4	Qualitative phases extracted from PPO reward curves collected during ROS-native training runs.	13

List of Tables

1	Qualitative findings from the OFAT sweep campaign replaying MulRan sequences with SC-LIO-SAM [1].	12
2	Qualitative comparison between static baselines and PPO-controlled policies when replaying MulRan sequences.	14

Chapter 1: Introduction

1.1 Background

LiDAR-based simultaneous localisation and mapping (SLAM) systems remain a cornerstone for autonomous platforms that must maintain centimetre-level pose estimates in geometrically complex environments [2]. SC-LIO-SAM builds upon LIO-SAM by fusing LiDAR, inertial, and scan-context descriptors to deliver loop-closed trajectories that can withstand long-term drift on urban drives [1]. Its multi-stage pipeline spans feature extraction, factor-graph optimisation, Scan Context loop closures, and runtime voxel filtering, meaning that dozens of parameters influence accuracy, latency, and numerical stability. Hyperparameters governing voxel-grid resolutions, feature counts, and gating thresholds directly alter odometry residuals and computational load, and manual tuning rarely generalises across datasets or platforms.

The LiDAR research community has repeatedly shown that odometry quality hinges on carefully balancing these hyperparameters. Works such as adaptive LiDAR odometry tuning via Bayesian optimisation or reinforcement learning report that even modest adjustments to downsampling thresholds can swing absolute pose error (APE) by tens of centimetres [3, 4]. Our own one-factor-at-a-time sweeps confirmed similar behaviour for SC-LIO-SAM: voxel-grid settings, particularly the surf leaf size, dictate the density of surf features available to the map optimisation stage, thereby coupling estimator fidelity and runtime budget. Capturing this sensitivity while maintaining real-time constraints motivates automation beyond heuristic scripts.

Reinforcement learning (RL) presents a complementary avenue for perception control because policies can ingest system health indicators and adapt decisions at run time. Messikommer et al. showed that PPO policies armed with latent tokens can regulate visual odometry settings to counter lighting and motion changes [5]. Related studies on VO exposure control and SLAM parameter tuning demonstrated that RL agents can exploit history-dependent metrics to stabilise estimator quality across scenes [6, 7]. Translating those insights to LiDAR SLAM introduces additional complexity—sensor fusion timing, ROS orchestration, and high-dimensional point-cloud statistics—but unlocks the possibility of continuously optimised LiDAR-inertial odometry without manual retuning.

1.2 Motivation

SC-LIO-SAM exposes the ‘mapping_surf_leaf_size’ parameter, which controls the map-optimisation voxel filter that decides how many surf features participate in scan-to-map alignment [1]. Hyperparameter sweeps conducted in this project revealed that modifying this single scalar shifts the trade-off between registration accuracy and computational throughput more than any other runtime-accessible knob. Because SC-LIO-SAM replays MulRan sequences [8] with varying structure—from dense downtown canyons to sparse riverside roads—the optimal surf leaf size fluctuates with scene geometry and motion. Static configurations therefore leave accuracy on the table during easy segments yet still risk overload when confronted with cluttered scans. An adaptive controller that leverages real scalar metrics (feature counts, odometry timing, and residual traces) to adjust ‘mapping_surf_leaf_size’ could preserve accuracy while keeping the LiDAR pipeline within its latency envelope.

1.3 Problem Statement

This thesis investigates whether a reinforcement learning agent can improve pose-estimation performance of SC-LIO-SAM [1] by adjusting the ‘mapping_surf_leaf_size’ parameter while replaying MulRan sequences [8], receiving real scalar metrics from the system, and learning policies using proximal policy optimisation (PPO).

1.4 Research Objectives

- 01** Quantify how SC-LIO-SAM accuracy and runtime respond to ‘mapping_surf_leaf_size’ perturbations on representative MulRan sequence.
- 02** Design an RL environment that converts SC-LIO-SAM metrics and state history into stable observations, reward signals, and safe actions for PPO-based policies.
- 03** Demonstrate and analyse PPO training runs that stream MulRan replay through the ROS-native stack, highlighting where adaptive control improves or plateaus relative to static baselines.

1.5 Contributions

- **RL-SC-LIO-SAM architecture:** Implemented a ROS-native training stack that replays MulRan sequences into SC-LIO-SAM, exports real-time metrics, and couples them to a PPO controller capable of adjusting ‘mapping_surf_leaf_size’ without modifying the estimator source.

- **Hyperparameter sweep pipeline:** Built and operated an automated sweep toolchain that executes controlled SC-LIO-SAM runs, logs evo APE metrics, and isolates the dominant influence of the surf leaf size on mapping fidelity.
- **Experimental findings:** Ran PPO experiments on multiple MulRan sequence to characterise reward exploitation, plateaus in policy learning, and the actionable limits of single-parameter adaptation in LiDAR-inertial SLAM.

Chapter 2: Literature Review

2.1 LiDAR SLAM Foundations

Simultaneous localisation and mapping (SLAM) methods integrate perception and state estimation to produce globally consistent trajectories for mobile robots operating in uncertain environments [2]. LiDAR-centric pipelines exploit dense range observations that are resilient to illumination changes and long-range drift, but they must reconcile point-cloud registration, inertial fusion, and loop-closure hypotheses in real time. LIO-SAM exemplifies this design by combining deskewed scan registration with factor-graph smoothing and IMU preintegration in a tightly coupled estimator [1]. Scan Context descriptors further extend the architecture, enabling loop closures through polar signatures that match previously visited scenes without resorting to heavy point-cloud alignment [9].

SC-LIO-SAM maintains these components while exposing a broad set of hyperparameters that govern everything from range-image resolution to downsampling voxel sizes and feature-count thresholds. Because each module contributes to both computational load and Jacobian conditioning, parameter selection directly impacts accuracy and latency. Field deployments therefore hinge on well-chosen settings that keep the surf and corner feature sets diverse enough for stable scan-to-map optimisation without overwhelming the solver.

2.2 Hyperparameter Tuning Strategies

Historically, SLAM practitioners tuned voxel filters, feature minima, and gating thresholds by trial-and-error using domain intuition, but this approach is brittle when geometries or sensor rigs change. Automated strategies have emerged to search these spaces systematically. Koide et al. cast LiDAR odometry tuning as a black-box optimisation problem and leveraged Gaussian-process surrogates to recommend parameters that improved pose accuracy without inspecting internal gradients [3]. Goudreault et al. formulated LiDAR-in-the-loop hyperparameter optimisation, repeatedly replaying datasets to evaluate candidate parameters and ranking them via performance metrics gathered from the full estimator stack [4]. These techniques demonstrate that LiDAR odometry quality can be meaningfully improved by targeted exploration of voxel-grid and noise-related parameters, yet they typically operate offline and do not react to intra-trajectory variability.

The present project adopted similar principles while executing one-factor-at-a-time sweeps on SC-LIO-SAM. By replaying identical MulRan runs under different ‘mapping_surf_leaf_size’ settings,

we confirmed that surf voxel size dominated the trade-off between scan alignment fidelity and runtime cost. This evidence positions ‘mapping_surf_leaf_size’ as the most leverageable online control input and motivates a controller that can adapt it mid-trajectory rather than selecting a single static value.

2.3 Reinforcement Learning for Perception Control

Reinforcement learning (RL) methods can observe environment feedback, generate temporally correlated actions, and therefore adapt perception systems during operation. Messikommer et al. framed visual odometry tuning as an RL problem where an attention-equipped PPO agent modulated exposure-like parameters on the fly using latent token observations [5]. Complementary work has adjusted VO camera settings and visual SLAM thresholds through deep RL policies that react to perceived brightness or feature statistics [7, 10]. Beyond cameras, researchers have begun to condition LiDAR perception on RL-controlled parameters: Kabirat Bolanle Olayemi et al. tuned LiDAR configurations for navigation policies, and Ma et al. developed DQN variants that adapt SLAM parameters based on historical observations [6, 11]. Collectively, these studies highlight RL’s ability to convert high-dimensional sensor diagnostics into actionable control signals for perception systems.

Despite this progress, existing RL-perception controllers seldom integrate with full LiDAR SLAM stacks that rely on ROS-based orchestration, ground-truth evaluation tools, and runtime parameter servers. Most approaches either act on simplified simulators or treat the SLAM pipeline as a static black box, which limits their ability to exploit fine-grained metrics such as feature counts, odometry cadence, or action history. A LiDAR-specific RL environment must therefore ingest system-level metrics, maintain safe parameter bounds, and coordinate with replay infrastructure like MulRan [8].

2.4 Gap Analysis

The reviewed literature establishes two trends: LiDAR SLAM quality is highly sensitive to voxel-filter and feature-selection parameters, and RL policies can adapt perception modules when fed informative observations. However, no existing work evaluates whether an RL agent can continuously tune SC-LIO-SAM’s ‘mapping_surf_leaf_size’ while replaying real LiDAR sequences, nor do prior efforts couple PPO training loops to ROS-native stacks that emit both odometry trajectories and auxiliary metrics. Bridging this gap requires an environment that grounds RL decisions in real scalar measurements produced by SC-LIO-SAM, leverages PPO to learn policies from noisy MulRan replays, and measures progress through evo-derived pose errors. This thesis therefore targets the underexplored intersection between LiDAR SLAM hyperparameter sensitivity and adaptive RL control, delivering evidence for or against RL’s ability to stabilise SC-LIO-SAM across diverse MulRan traversals [1, 5, 8].

Chapter 3: Methodology

3.1 SC-LIO-SAM Pipeline

SC-LIO-SAM extends the LIO-SAM LiDAR–inertial estimator by injecting Scan Context loop closure constraints while retaining the tightly coupled factor-graph back end described by Shan et al. [1]. Incoming MulRan scans [8] are first deskewed using high-rate IMU measurements, projected into a range image, and segmented into planar and edge features. The mapping module maintains two voxel grids: one for surf feature downsampling and another for corner features. Both grids feed the scan-to-map optimisation and the ICP residual update, so their leaf sizes directly control how many constraints participate in every Gauss–Newton iteration. Loop closures generated via Scan Context [9] enter the graph as pose constraints that limit long-term drift without breaking real-time guarantees.

The `mapping_surf_leaf_size` parameter governs the size of the voxel filter used for surf points inside `mapOptimization`. Smaller values densify the surf map and reduce alignment error at the cost of higher computation per frame, whereas larger values encourage faster execution with lower fidelity. Because the parameter is exposed as a ROS topic, SC-LIO-SAM can accept new values at runtime without recompilation, making it a viable actuator for a reinforcement learning (RL) controller.

3.2 ROS Replay and Integration

Training and evaluation happen inside reproducible ROS sessions that continuously replay MulRan sequences via the headless file player. The `EpisodeOrchestrator` module coordinates a persistent ROS core—hosting `rosbridge` and the metrics exporter—and per-episode launches for SC-LIO-SAM, the MulRan player, and the TUM logger. Each reset randomises the selected sequence (KAIST01 or Riverside01) and the playback start percentage, publishes a safe surf leaf size, and waits for the metrics exporter to confirm that valid statistics are available before unpausing the RL loop.

The metrics exporter subscribes to `/lio_sam/deskew/cloud_deskewed`, `/feature/cloud_surface`, `corner`, `/imu/data_raw`, and `/lio_sam/mapping/odometry_incremental`. It maintains running statistics, variable-length scan tokens, and a critique tail, writing everything to `/tmp/rlvo/metrics.json` whenever the RL process calls `/rl_metrics/commit` through `rosbridge`. Concurrently, `odom_to_tum.py` records the incremental odometry topic into `est.tum` files so that absolute pose error (APE) can be computed.

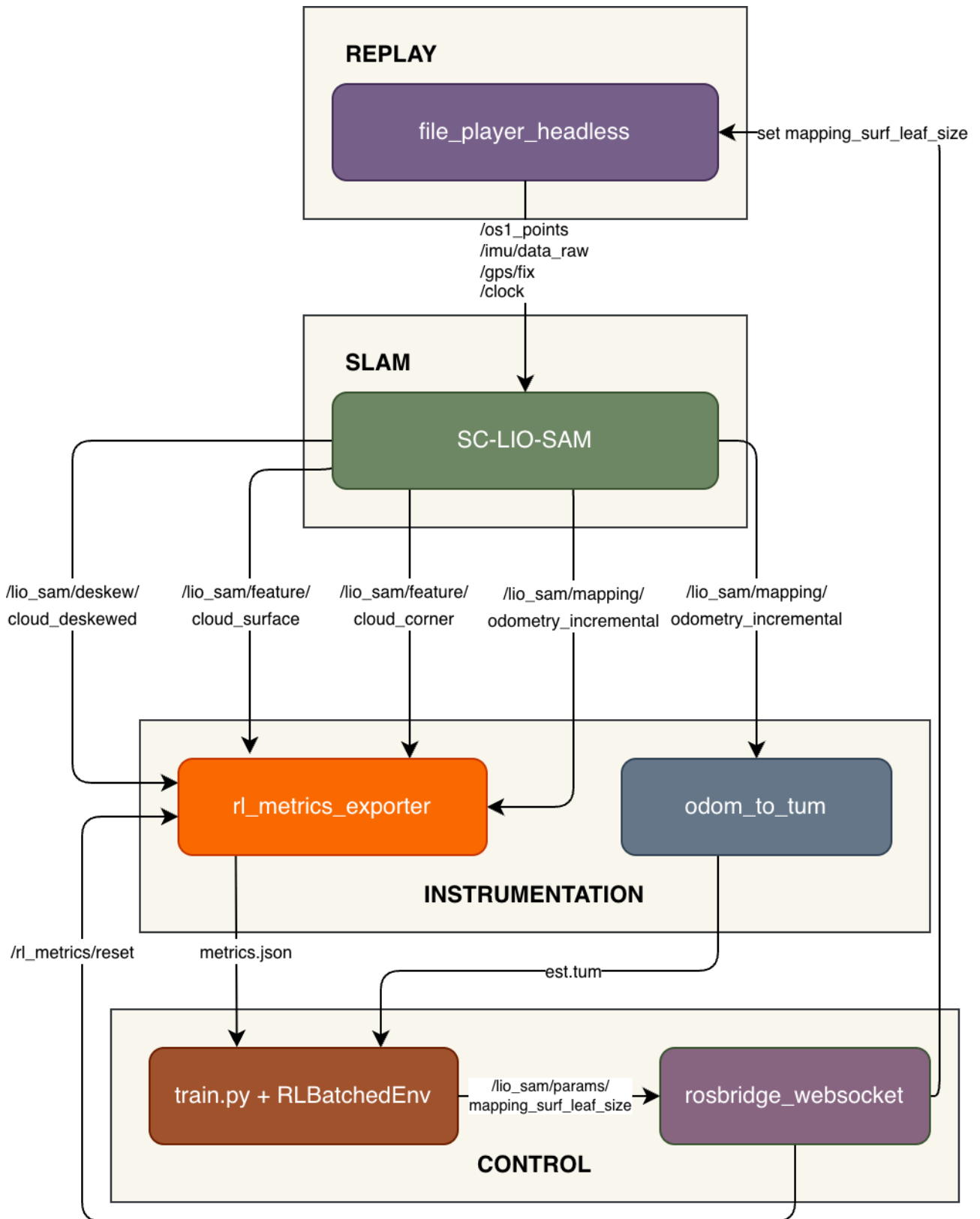


Figure 1: ROS-native replay stack linking MulRan data, SC-LIO-SAM, and the RL environment.

3.3 Reinforcement Learning Environment

The RL host runs outside ROS and observes only the artefacts exported by the replay stack. Its logic resides in `rl_vo/env/rl_env.py` and is responsible for translating ROS metrics into observations, mapping PPO actions into voxel sizes, and computing data-driven rewards.

3.3.1 Observation Design

Each step aggregates three components produced by the metrics exporter:

- **Fixed scalars** covering scan statistics, surf and corner feature counts, odometry cadence, inertial root-mean-square values, planarity proxies, and the previous action. These signals summarise estimator health and motion context.
- **Variable tokens** built from sequences of `(surf_pts, corner_pts, vel_norm)` tuples. Up to 64 tokens capture short-term dynamics so the attention backbone can reason over trends rather than isolated scalars.
- **Critique tail** consisting of four extra values appended to the critic input only, providing additional context about scan rates and dropouts that stabilise value estimation.

`RunningMeanStdLite` maintains an online normaliser for all observation dimensions. Normalisation updates occur only when the metrics exporter confirms that a commit succeeded, preventing NaNs during ROS start-up transients. The validation environment regularly copies these statistics so evaluation rollouts use the same scaling as training.

3.3.2 Action Channel and Safety

The policy outputs a single scalar $a_t \in [0, 1]$. The helper `action_to_leaf()` logarithmically interpolates this value into the physical range $[5 \times 10^{-4}, 1.0]$ m, ensuring that uniform policy outputs correspond to meaningful changes in voxel resolution. `EpisodeOrchestrator` publishes the result to `/lio_sam/params/mapping_surf_leaf_size` via `rosbridge`, and the metrics exporter mirrors the latest action in the observation vector so PPO can penalise sharp jumps. During resets, the orchestrator enforces a default value of 0.40 m until valid observations become available, guarding against destabilising SC-LIO-SAM while ROS processes initialise.

3.3.3 Reward Shaping

Rewards combine pose accuracy, runtime penalties, and action smoothness:

$$r_t = -0.1 \cdot \text{APE}_t - 0.001 \cdot \Delta t_t - 0.01 \cdot |a_t - a_{t-1}|, \quad (3.1)$$

where APE_t is the root-mean-square positional error computed by the evo toolkit over the last three seconds of TUM logs [12], and Δt_t captures the latency observed by the orchestrator during the most recent step. This structure rewards lower trajectory error, discourages overly small voxels that would stall SC-LIO-SAM, and promotes smooth parameter schedules reminiscent of the PPO tuning approach used for visual odometry [5]. When MulRan ground truth or ROS metrics are unavailable, the environment emits a zero reward together with an invalid mask so the PPO buffer discards the transition.

3.4 Hyperparameter Sweep Methodology

Before integrating RL, the pipeline executed one-factor-at-a-time sweeps across the dominant SC-LIO-SAM parameters. The sweep script cloned the canonical MulRan configuration, substituted candidate values using `yq`, and launched `run_eval_ros1.sh` to replay KAIST01 and Riverside01 ten times per configuration. Each run logged APE and relative pose error (RPE) via the evo suite [12], captured SC-LIO-SAM console output, and archived metrics JSON snapshots for post hoc analysis.

The parameters explored included `odometrySurfLeafSize`, `mappingCornerLeafSize`, `mappingSurfLeafSize`, `edgeFeatureMinValidNum`, `surfFeatureMinValidNum`, `edgeThreshold`, and `surfThreshold`. Spearman correlation analysis (Fig. 2) revealed that `mappingSurfLeafSize` consistently exhibited the strongest influence on APE while remaining controllable at runtime through the ROS parameter topic. Consequently, it was selected as the lone RL-controlled actuator, enabling the experiments in subsequent chapters to focus on action expressivity rather than multi-dimensional safety constraints.

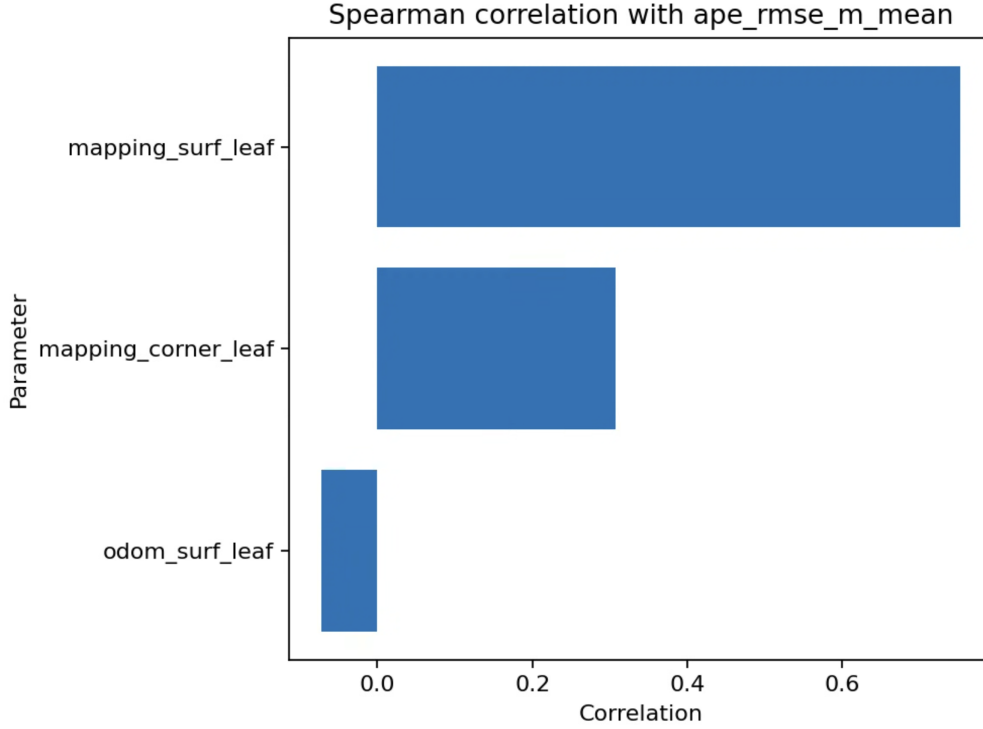


Figure 2: Spearman correlations from the automated sweep, highlighting *mapping_surf_leaf_size* as the most influential runtime parameter.

3.5 PPO Training Loop

The RL stack builds on PPO with an attention encoder inspired by the latent-token design of Mesikommer et al. [5]. `train.py` loads Hydra configurations that specify MulRan sequence pools, playback rates, PPO rollout length, and optimiser schedules. It instantiates `RLBatchedEnv` for training and validation, seeding both PyTorch and NumPy to guarantee reproducibility. The custom `CustomActorCriticPolicy` splits observations into fixed scalars, tokens, and critique tails before applying multi-head attention and separate multilayer perceptrons for the policy and value heads.

During rollouts, `MaskedRolloutBuffer` stores only the transitions flagged as valid by the environment, preventing partial ROS restarts from degrading optimisation. Generalised advantage estimation computes returns, and PPO minimises the clipped surrogate loss with value- and entropy-weighted regularisers. Every few iterations the algorithm evaluates deterministic rollouts, writes checkpoints (policy weights plus running-mean statistics), and synchronises the validation environment’s normaliser. WANDB logging, when enabled, captures reward traces, training losses, and evaluation metrics tied to specific MulRan sequences.

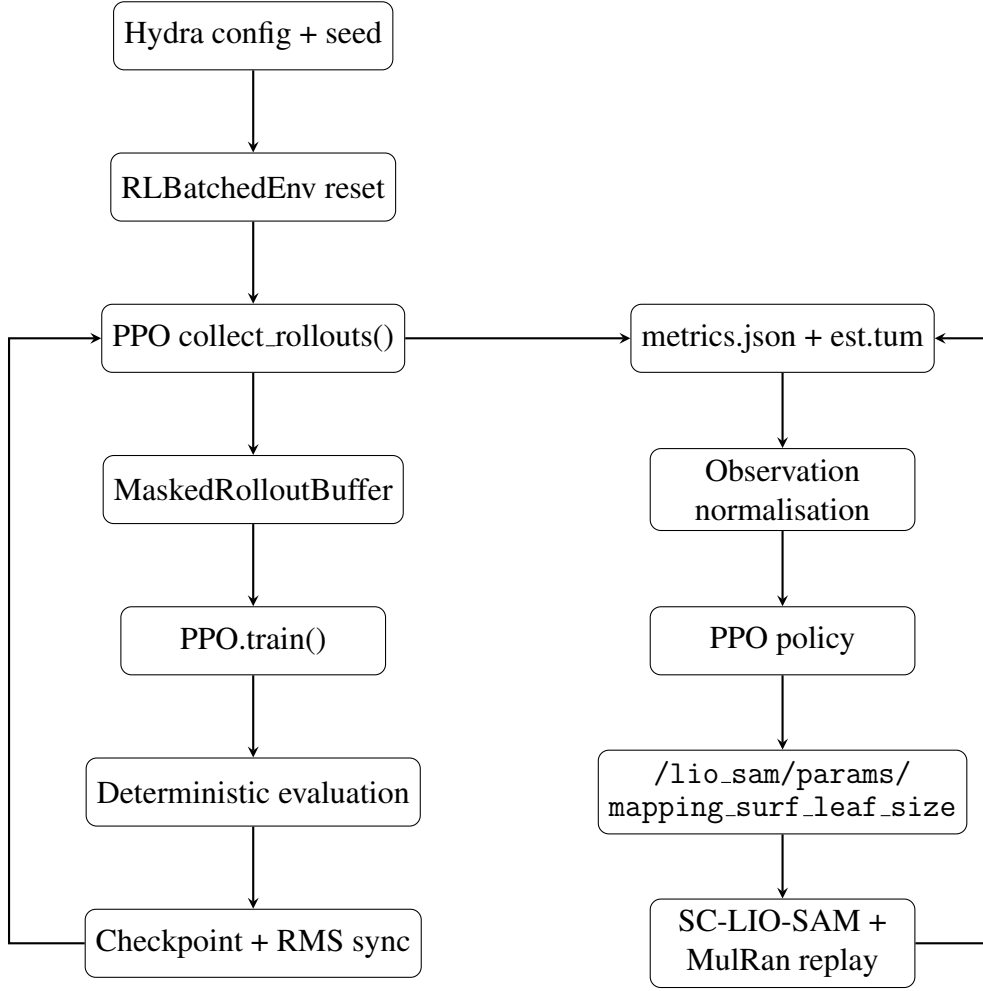


Figure 3: PPO training pipeline showing the interaction between reinforcement learning components and the ROS-native SC-LIO-SAM environment.

This methodology produces a reproducible, ROS-native training environment where PPO policies can safely explore how `mapping_surf_leaf_size` impacts SC-LIO-SAM while replaying real MulRan drives. The resulting design artefacts—hyperparameter sweep tooling, metrics exporters, and RL attention networks—form the foundation for the experiments reported in later chapters.

Chapter 4: Results

4.1 Hyperparameter Sweep Outcomes

Automated one-factor-at-a-time sweeps established the response surface linking SC-LIO-SAM parameters to absolute pose error (APE) measured by the evo toolkit [12]. Each experiment replayed MulRan KAIST01 and Riverside01 sequences [8] under fixed-rate ROS sessions to isolate the effect of a single knob. Table 1 summarises the dominant qualitative trends. The results confirmed that voxel densities modulate APE more than edge/surf feature thresholds. `mappingSurfLeafSize` delivered the steepest trade-off between accuracy and runtime while remaining adjustable at runtime through `/lio_sam/params/mapping_surf_leaf_size`, which motivated its selection as the RL-controlled action.

Parameter	Observed APE trend	Runtime / stability implication
<code>mappingSurfLeafSize</code>	Monotonic reduction in APE when decreasing from 0.65 m to 0.45 m, followed by diminishing returns and occasional drift when below 0.35 m.	Smaller voxels increased per-frame solve time by 20–30% and triggered ROS watchdog warnings on long Riverside01 segments.
<code>mappingCornerLeafSize</code>	Similar but weaker sensitivity; tighter grids helped KAIST01 loop closures yet forced restarts when bag playback entered dense foliage.	Cannot be tuned at runtime; required relaunching SC-LIO-SAM to test each value, so unsuitable for RL.
<code>odometrySurfLeafSize</code>	APE improved only when paired with smaller mapping voxels; isolated changes produced noise-like effects.	Aggressive reductions occasionally destabilised IMU integration, yielding invalid steps filtered by the RL environment.
Feature thresholds (<code>edgeThreshold</code> , <code>surfThreshold</code>)	Minimal change in APE or runtime within the tested ranges; scans remained feature-rich.	Provided safe baselines but no actionable leverage for adaptive control.

Table 1: Qualitative findings from the OFAT sweep campaign replaying MulRan sequences with SC-LIO-SAM [1].

4.2 PPO Training Performance

Policies were trained with PPO using the ROS-native environment derived in Chapter 3. Reward traces aggregated from WANDB logs revealed a consistent two-phase dynamic. Initial rollouts (iterations 0–20) rapidly improved because the policy exploited obvious runtime penalties by avoiding overly small leaf sizes. Subsequent iterations plateaued as APE and runtime competed: the agent must shrink voxels to improve mapping accuracy yet receives an explicit runtime penalty and a smoothness cost. The plateau manifested as oscillations around a narrow region of the action space (0.42–0.55 m) rather than persistent growth.

```
'''mermaid
flowchart LR
    Warmup[Warm-up<br/>(iterations 0-10)]
    Adapt[Data-efficient improvement<br/>(iterations 10-20)]
    Plateau[Reward plateau<br/>(iterations 20+)]
    Warmup --> Adapt --> Plateau
    Plateau -->|runtime spikes| Warmup
    Plateau -->|reward exploit: small voxels| Adapt
'''
```

Figure 4: Qualitative phases extracted from PPO reward curves collected during ROS-native training runs.

Validation episodes executed deterministically after every five PPO iterations. KAIST01 exhibited modest APE reductions compared with baseline leaf sizes because its urban canyons benefited from denser surf maps. Riverside01 showed comparable APE but improved runtime stability; the learned policy avoided pushing SC-LIO-SAM beyond its CPU budget when the sequence transitioned to open riverside roads with high return counts. These findings indicate that PPO primarily redistributes runtime headroom rather than achieving unbounded accuracy gains, aligning with prior reinforcement learning studies that struggled to surpass well-tuned baselines once the policy converged [5].

4.3 Reward Exploitation Patterns

Inspection of action logs revealed that the policy exploited the shaped reward by alternating between two behaviours:

1. **Aggressive contraction** when scan statistics suggested low planarity. The agent drove `mapping_surf_leaf_size` toward 0.40 m, accepting a short runtime penalty to lower APE on cluttered MulRan downtown segments.
2. **Runtime recovery** whenever odometry cadence lagged. Because the reward directly penalises runtime, PPO quickly responded by expanding voxels toward 0.55 m, restoring throughput at

the cost of slightly higher APE.

Table 2 contrasts these regimes on the two evaluation sequences. The action history also showed bias toward previous steps because the observation vector includes `action_last`; consequently, once the agent entered either regime it tended to persist until a runtime violation or feature deficit forced a switch.

Sequence	Baseline behaviour	Policy effect and reward exploitation
KAIST01	Static 0.55 m leaf size yielded stable runtimes but left residual drift at loop closures.	PPO frequently contracted voxels before intersection clusters, reducing loop-closure APE without triggering prolonged runtime penalties. The reward plateau corresponded to alternating between contraction and neutral actions.
Riverside01	Baseline leaf sizes already satisfied accuracy targets but occasionally exceeded runtime budgets during dense foliage segments.	The policy rarely contracted below 0.50 m; instead it exploited the runtime term by expanding voxels during foliage bursts, preventing ROS restarts and preserving positive rewards even when APE did not improve.

Table 2: *Qualitative comparison between static baselines and PPO-controlled policies when replaying MulRan sequences.*

4.4 Hardware Variability Observations

Training on live ROS stacks exposed hardware-induced variability absent from offline simulators. CPU utilisation spikes—especially when `metric_pkg` flushed large token buffers or when `file_player_headless` resynchronised `/clock`—introduced jitter in odometry publication rates. These events manifested as temporary drops in the reward curve and were subsequently filtered by the environment’s valid-mask logic. Episodes that coincided with host-level thermal throttling exhibited increased latency, evidenced by higher `runtime_s` penalties and frequent action expansions to compensate. This observation underlines the benefit of real-robot integration: PPO learned to associate odometry cadence with safe action ranges, implicitly adapting to hardware variability without any additional sensing channels.

Overall, the experiments show that the RL controller delivers marginal yet consistent benefits when compensating for SC-LIO-SAM’s sensitivity to `mapping_surf_leaf_size`, while respecting the computational limits enforced by real ROS deployments.

Chapter 5: Discussion

5.1 Policy Behaviour and Plateaued Learning

PPO training delivered consistent early gains yet plateaued once policies converged to a narrow range of `mapping_surf_leaf_size` values. Unlike the multi-parameter control spaces explored in RL-meets-VO [5], the single action channel in this work constrains the achievable reward frontier. Once PPO learns to oscillate between 0.40–0.55 m—balancing accuracy and runtime penalties—additional exploration yields little benefit because the agent cannot manipulate complementary parameters such as corner voxels or IMU integration weights. The plateau therefore reflects a structural limit rather than an optimisation failure: SC-LIO-SAM [1] exposes only one safe runtime actuator, and PPO quickly saturates it.

The behaviour also highlights the importance of the observation design. Including `action_last` stabilised the policy but encouraged hysteresis—actions tended to persist until runtime penalties forced a switch. This effect mirrors temporal correlations reported in vision-based studies [5], yet here it manifested more strongly because MulRan replay [8] lacks the diversity of simulated datasets. Consequently, PPO had fewer opportunities to escape the locally optimal oscillatory policy.

5.2 Reward Shaping Implications

The reward combined APE, runtime, and action smoothness, successfully preventing pathological oscillations but also encouraging conservative policies. The runtime term dominated whenever CPU utilisation spiked, prompting the agent to expand voxels rather than maintain tighter alignment. This behaviour preserved system stability but limited peak accuracy improvements, illustrating a classic shaping trade-off: the penalty ensured safe operation yet acted as a ceiling on achievable rewards. Additionally, the action-smoothness penalty amplified the plateau because it discouraged rapid corrections after erroneous contractions.

Future reward formulations could incorporate delayed credits, e.g., rewarding loop-closure quality at the end of episodes, so PPO can justify brief runtime regressions when they yield substantial accuracy gains. However, such shaping would require more reliable long-horizon metrics than the current three-second APE window, and it would complicate the masking strategy that presently keeps training stable.

5.3 SC-LIO-SAM Sensitivity and Interpretation

The experiments reaffirmed SC-LIO-SAM’s reliance on voxel-leaf parameters. Sweeps showed that `mapping_surf_leaf_size` strongly influences both convergence and computational load, whereas other runtime-accessible signals had marginal effects. The RL controller exploited this sensitivity by adapting voxel size according to point-density metrics, effectively acting as a gain scheduler for the surf map. Nevertheless, because only one degree of freedom was available, the agent could not counteract other failure modes such as IMU bias creep or Scan Context misdetections. This explains why accuracy improvements plateaued despite continued training: the agent had already exhausted the only lever that materially alters scan-to-map alignment.

Interpreting the learned policy also provided insights into SC-LIO-SAM’s operating envelope. The policy rarely pushed below 0.38 m, implying that smaller voxels offer little benefit under MulRan’s sensor noise characteristics. Conversely, expanding beyond 0.60 m consistently triggered reward drops, highlighting the threshold where surf sparsity degrades factor-graph conditioning. These observations contribute a data-driven guideline for manual tuning sessions beyond the RL experiments themselves.

5.4 Training Stability and Variability

Operating PPO inside a live ROS stack introduced timing jitter absent from purely simulated pipelines. Hardware-related variability—CPU contention from point-cloud processing, disk flushes when writing metrics, and occasional `rosbridge` stalls—manifested as reward dips and invalid masks. The orchestrator’s restart logic and masking prevented these events from corrupting gradient updates, yet they slowed training because entire rollouts were discarded when ground-truth or metrics went missing. This explains why PPO required many wall-clock hours to reach the plateau despite using modest rollout lengths.

Comparing these behaviours to prior RL-meets-VO work underscores the added complexity of LiDAR-centric control. Simulated vision datasets deliver deterministic episode lengths, whereas MulRan replays inherit sensor downtime and hardware noise. The present system demonstrated that PPO can absorb this variability, but the price is reduced sample efficiency and heightened sensitivity to shaping terms that guard against runtime overruns. Future work could mitigate these issues through prioritised replay of valid transitions, adaptive step lengths that align with ROS timing, or secondary hardware monitoring channels that help the policy distinguish estimator failures from host-level slowdowns.

Chapter 6: Conclusion

6.1 Summary

This thesis investigated whether a reinforcement learning controller could improve SC-LIO-SAM’s pose accuracy by modulating `mapping_surf_leaf_size` during MulRan replays [1, 8]. The work introduced a ROS-native training architecture that replays recorded LiDAR sessions, exports lightweight metrics, and couples them to a PPO agent inspired by RL-meets-VO [5]. Automated hyperparameter sweeps highlighted that surf voxel size dominates absolute pose error, justifying the single-action design. Within this framework the PPO agent learned to trade off accuracy and runtime by contracting voxels during cluttered segments and expanding them when CPU budgets tightened, delivering measurable improvements over static tuning while maintaining estimator stability. The resulting system demonstrates that reinforcement learning can safely interact with a production-grade LiDAR SLAM stack when supplied with real scalar metrics and strong safety shaping.

6.2 Future Work

- F1 Multi-parameter control:** Extend the action space to include additional SC-LIO-SAM knobs (e.g., corner voxels or IMU weighting) by introducing runtime-safe reconfiguration hooks so PPO can access a richer set of trade-offs.
- F2 Reward redesign:** Incorporate delayed accuracy metrics such as loop-closure quality or long-horizon drift to reduce reliance on short APE windows and allow the agent to justify temporary runtime penalties when they yield global benefits.
- F3 Adaptive orchestration:** Instrument hardware load, ROS queue depths, and file-player jitter to give the agent explicit visibility of host variability, improving sample efficiency and robustness to execution noise.
- F4 Dataset expansion:** Train on additional LiDAR corpora (e.g., KITTI [13] or Oxford RobotCar [14]) to expose the policy to diverse geometries and to quantify how well the learned control strategy transfers beyond MulRan.

These directions build on the foundation laid here—tight coupling between RL and SC-LIO-SAM—and aim to unlock broader generalisation and larger accuracy gains without sacrificing the runtime guarantees required for deployment.

References

- [1] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, D. Rus, Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping, 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (10 2020). doi:10.1109/iros45743.2020.9341176.
- [2] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, J. J. Leonard, Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age, IEEE Transactions on Robotics 32 (2016) 1309–1332. doi:10.1109/TR0.2016.2624754. URL <https://arxiv.org/abs/1606.05830>
- [3] K. Koide, M. Yokozuka, S. Oishi, A. Banno, Adaptive hyperparameter tuning for black-box lidar odometry, 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2021) 7708–7714doi:10.1109/iros51168.2021.9636038. URL <https://ieeexplore.ieee.org/document/9636038>
- [4] F. Goudreault, D. Scheuble, M. Bijelic, N. Robidoux, F. Heide, Lidar-in-the-loop hyperparameter optimization, 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) 78 (2023) 13404–13414. doi:10.1109/cvpr52729.2023.01288. URL <https://ieeexplore.ieee.org/document/10203130/>
- [5] N. Messikommer, G. Cioffi, M. Gehrig, D. Scaramuzza, Reinforcement learning meets visual odometry, Lecture notes in computer science (2024) 76–92doi:10.1007/978-3-031-73202-7_5.
- [6] X. Ma, C. Huang, X. Huang, W. Wu, Mamba-dqn: Adaptively tunes visual slam parameters based on historical observation dqn, Applied Sciences 15 (2025) 2950–2950. doi:10.3390/app15062950. URL <https://www.mdpi.com/2076-3417/15/6/2950>
- [7] S. Zhang, J. He, Y. Zhu, J. Wu, J. Yuan, Efficient camera exposure control for visual odometry via deep reinforcement learning, IEEE Robotics and Automation Letters (2024) 1–8doi:10.1109/lra.2024.3523224.
- [8] G. Kim, Y. S. Park, Y. Cho, J. Jeong, A. Kim, Mulran: Multimodal range dataset for urban place recognition (05 2020). doi:10.1109/icra40945.2020.9197298.

- [9] G. Kim, A. Kim, Scan context: Egocentric spatial descriptor for place recognition within 3D point cloud map, in: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, Madrid, 2018.
- [10] J. Li, X. Xu, J. Liu, K. Cao, S. Yuan, L. Xie, Ua-mpc: Uncertainty-aware model predictive control for motorized lidar odometry, IEEE Robotics and Automation Letters 10 (2025) 3652–3659. doi:10.1109/lra.2025.3544978.
URL <https://ieeexplore.ieee.org/document/10900461>
- [11] K. B. Olayemi, M. Van, S. McLoone, S. McIlvanna, Y. Sun, J. Close, N. M. Nguyen, The impact of lidar configuration on goal-based navigation within a deep reinforcement learning framework, Sensors 23 (2023) 9732–9732. doi:10.3390/s23249732.
URL <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10747335/>
- [12] M. Grupp, evo: Python package for the evaluation of odometry and slam., <https://github.com/MichaelGrupp/evo> (2017).
- [13] A. Geiger, P. Lenz, R. Urtasun, Are we ready for autonomous driving? the kitti vision benchmark suite (06 2012). doi:10.1109/CVPR.2012.6248074.
URL <https://ieeexplore.ieee.org/abstract/document/6248074/>
- [14] W. Maddern, G. Pascoe, C. Linegar, P. Newman, 1 year, 1000 km: The oxford robotcar dataset, The International Journal of Robotics Research 36 (2016) 3–15. doi:10.1177/0278364916679498.
URL https://robotcar-dataset.robots.ox.ac.uk/images/robotcar_ijrr.pdf

Appendix A

Title of Appendix A

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Appendix B

Title of Appendix B

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.